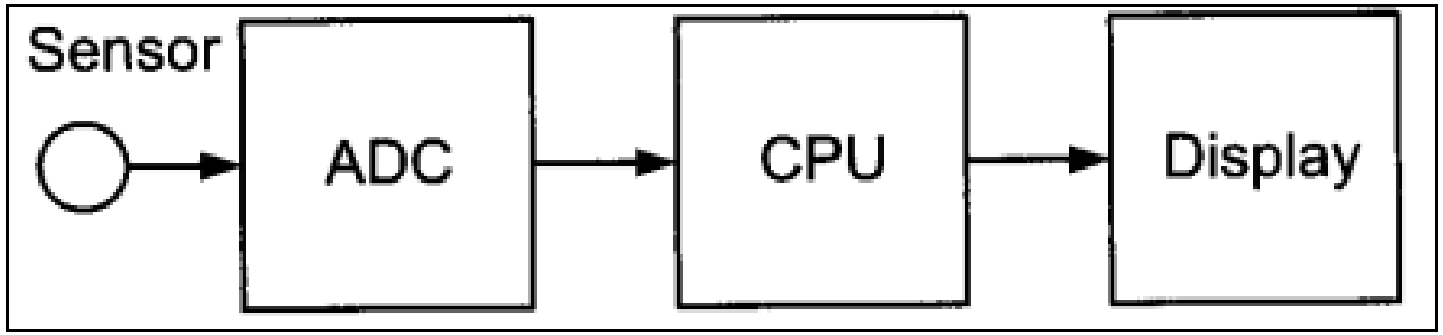
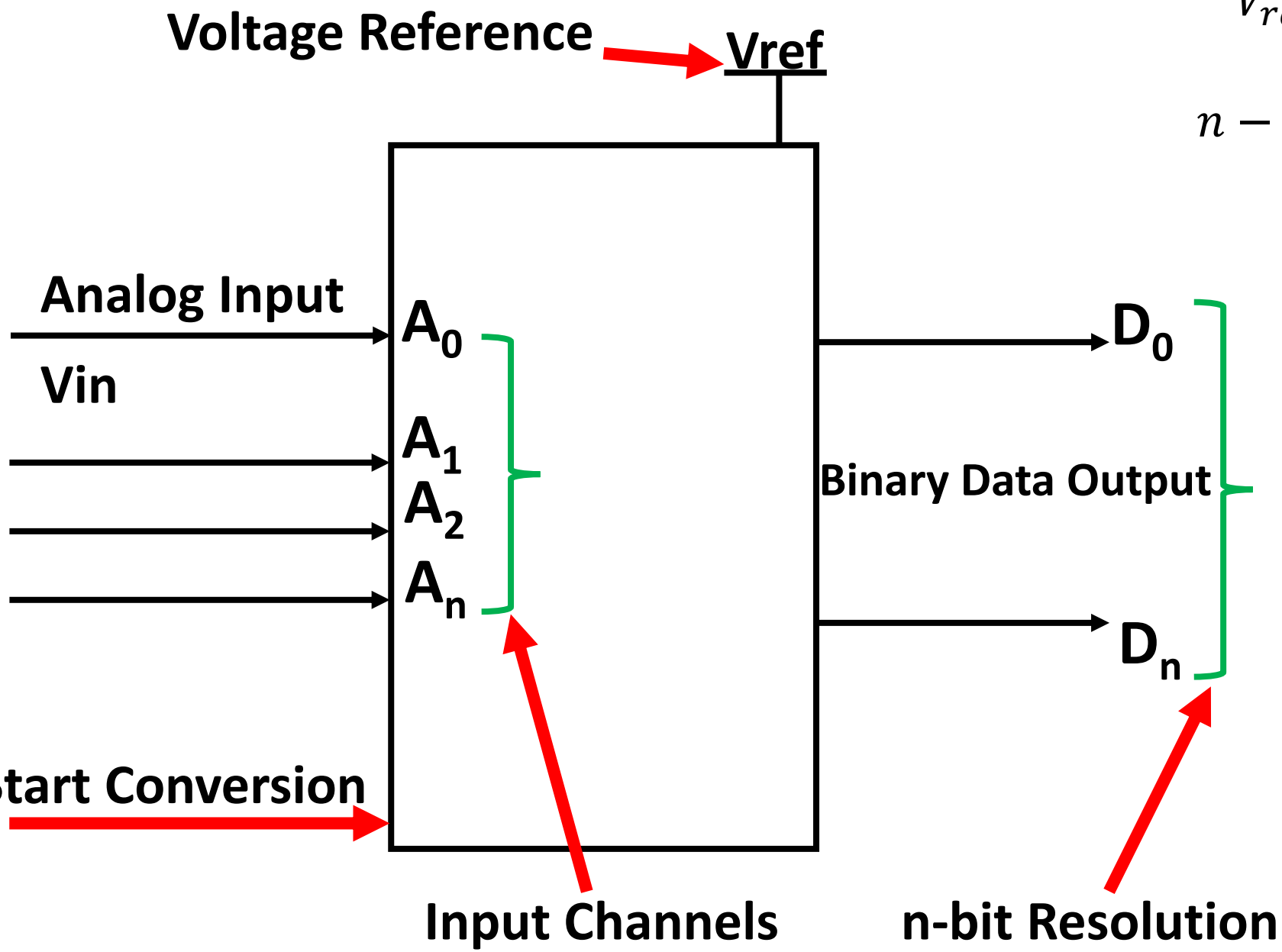
Analog to Digital Converter (ADC)

Digital Computers use binary (discrete) values, physical world is analog (continuous).
Physical quantities dealt in every day life are temperature, heat, light, pressure (wind or liquid), humidity, flow rate and velocity.
A physical quantity is converted to electrical (voltage, current) signals using a device called a transducer (sensor).
ADC are required to convert real world physical quantities into digital numbers so that microcontrollers (computers) can read, process and store them.
ADCs are used for data acquisition.



Analog to Digital Converter (ADC)



$V_{ref} \Rightarrow Sets\ the\ input\ range$

$\Rightarrow Sets\ the\ V_{res}$

$n - bit \Rightarrow Sets\ the\ V_{res}$

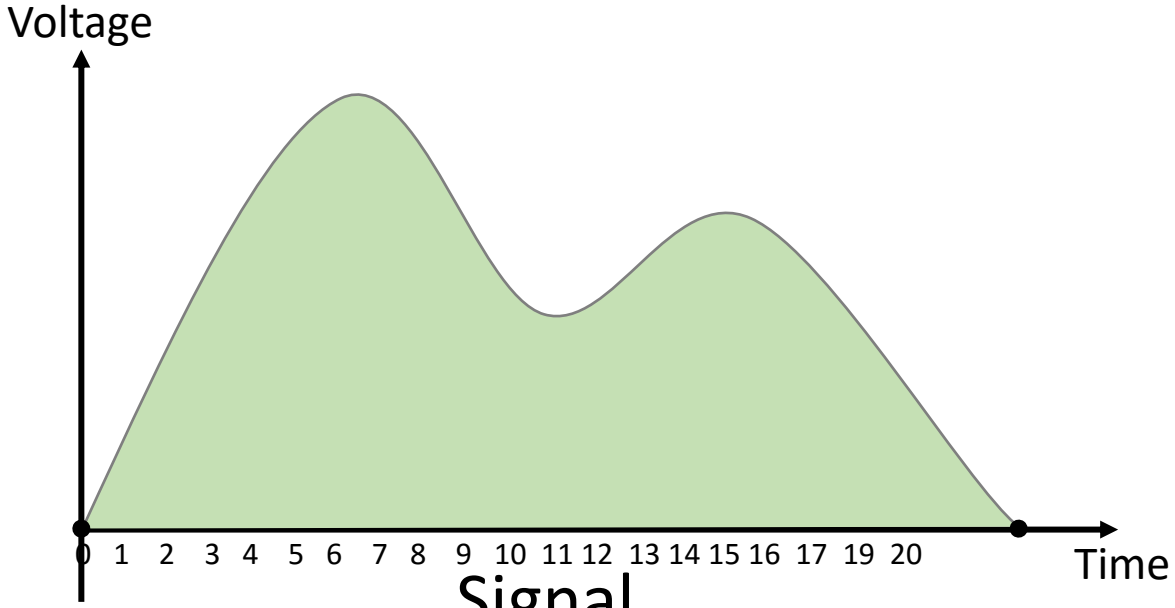
$$V_{res} = \frac{V_{ref}}{2^n}$$

Problem:

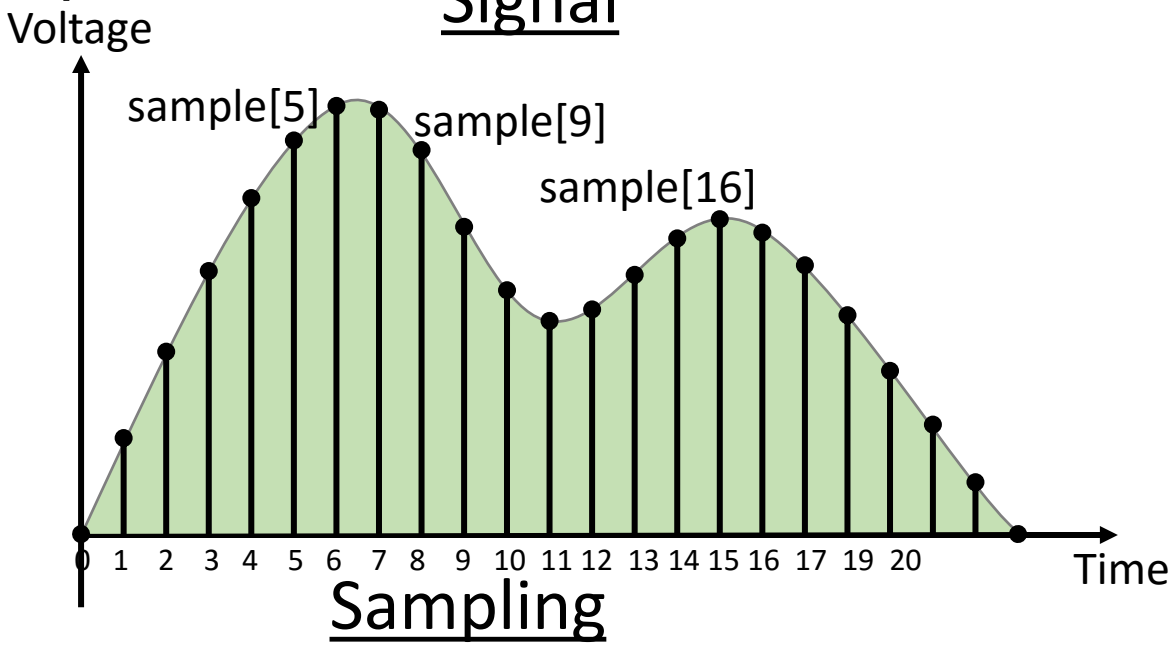
$$V_{res} = \frac{V_{ref}}{2^n - 1}$$

Do not use it for finding V_{res}

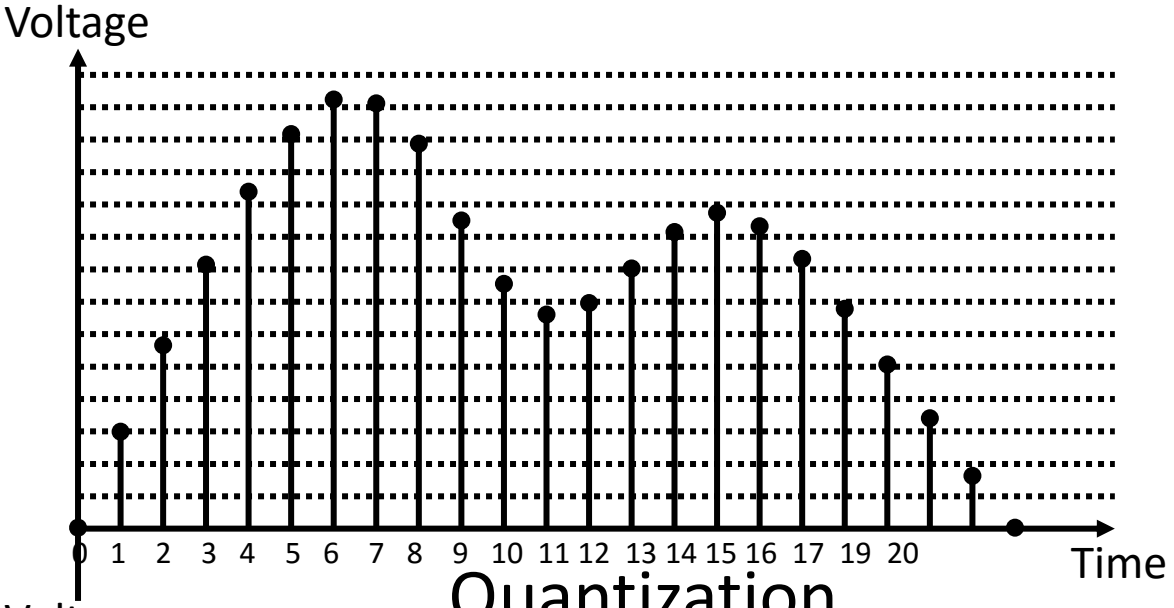
Digital Instrumentation (Sampling and Quantization)



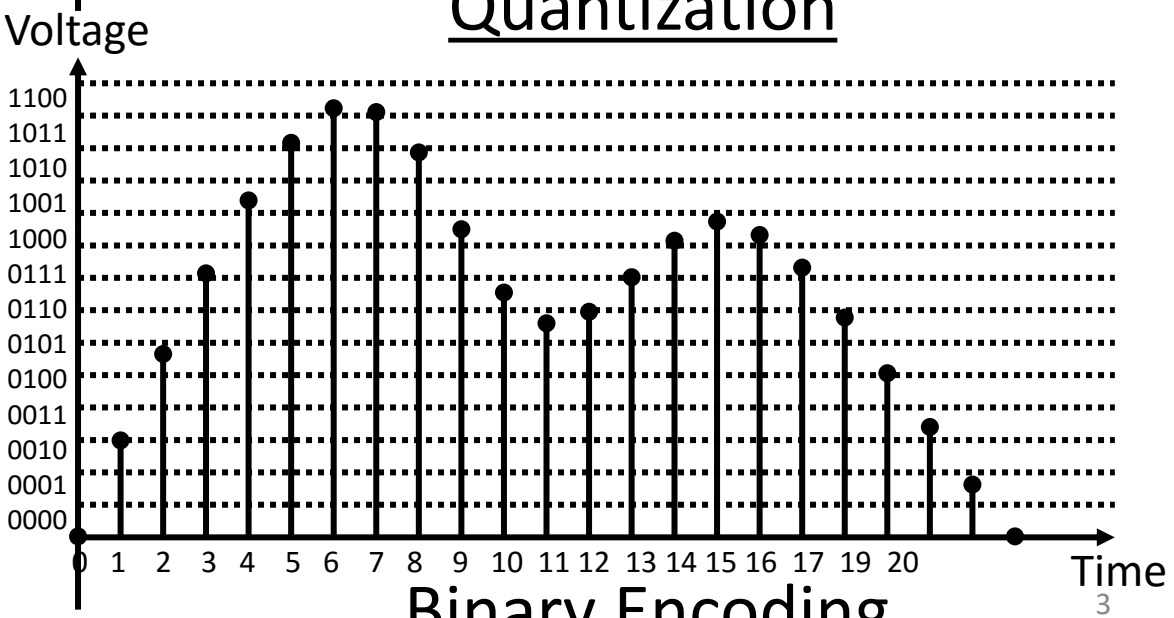
Signal



Sampling

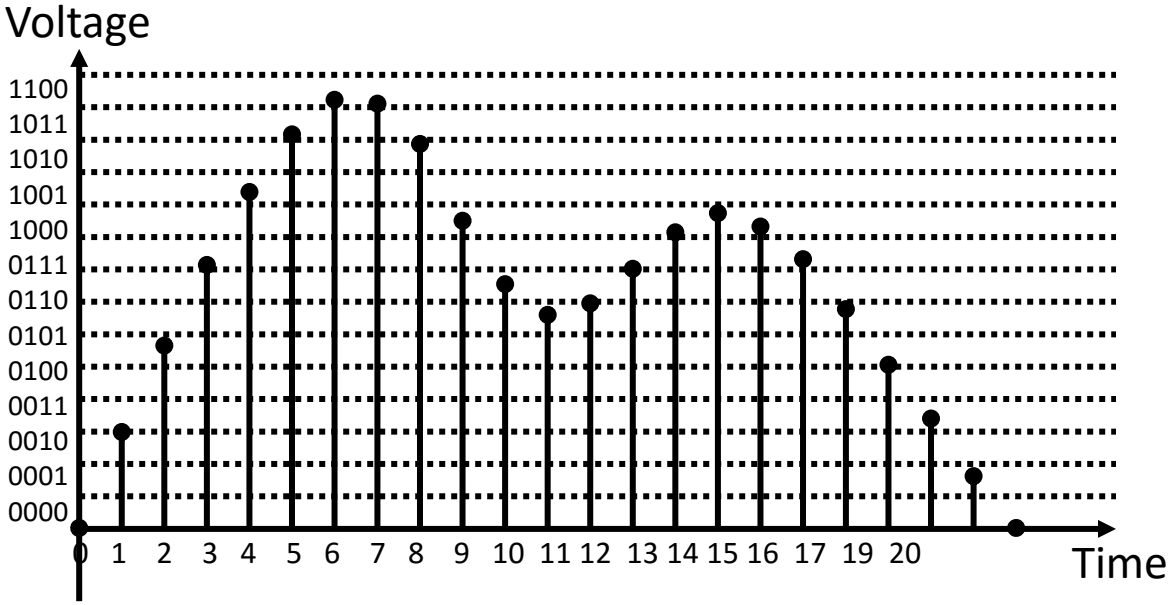


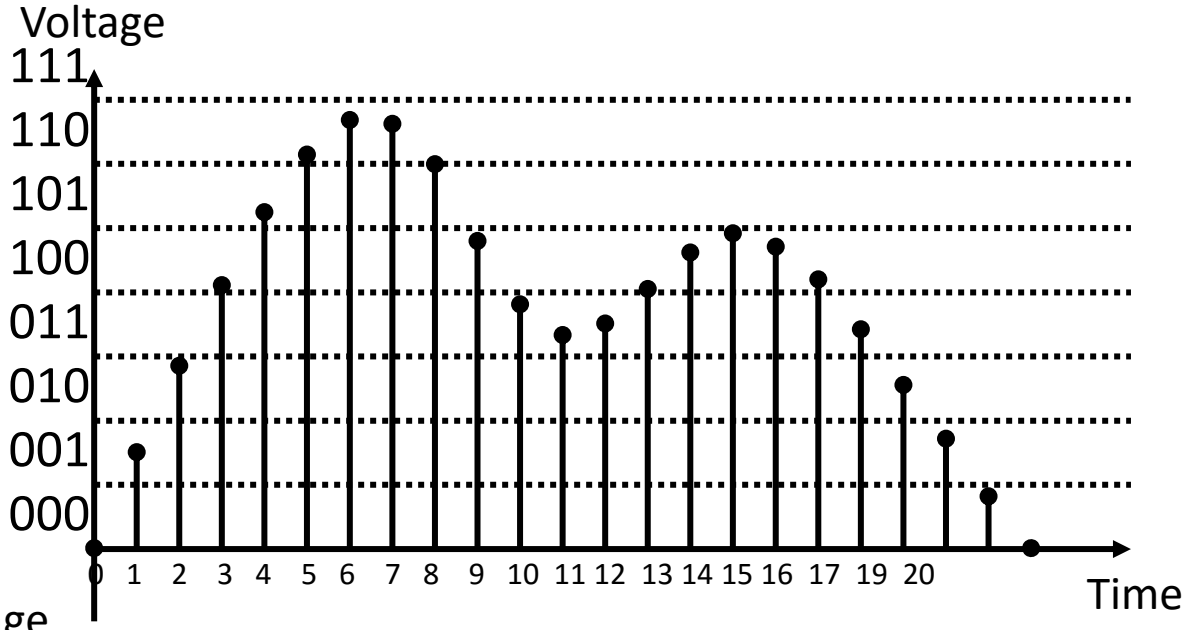
Quantization

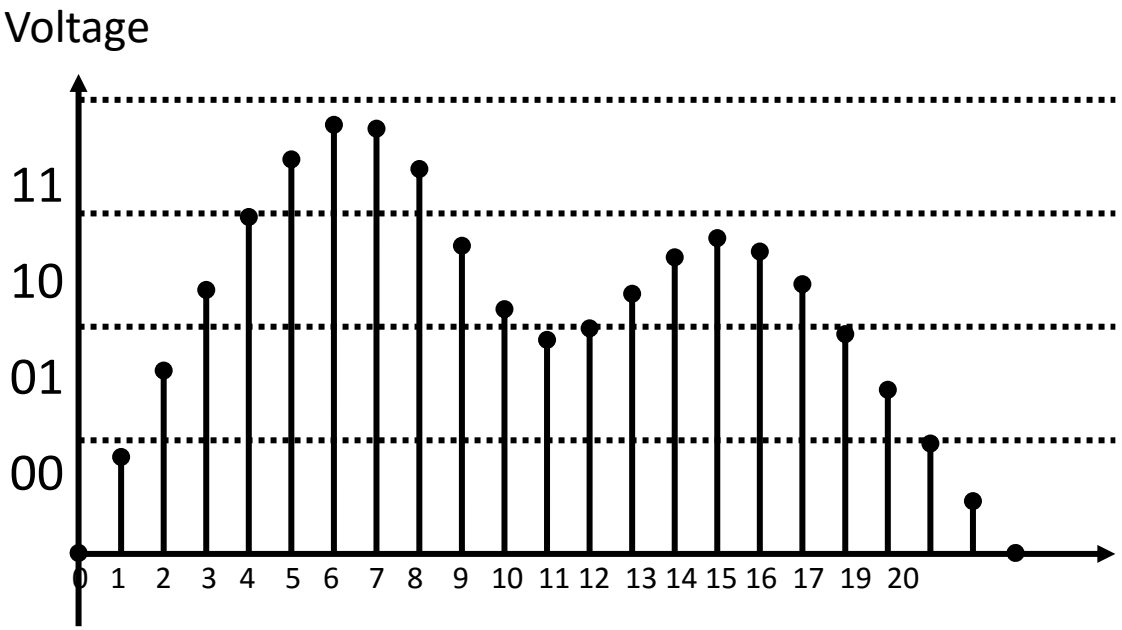


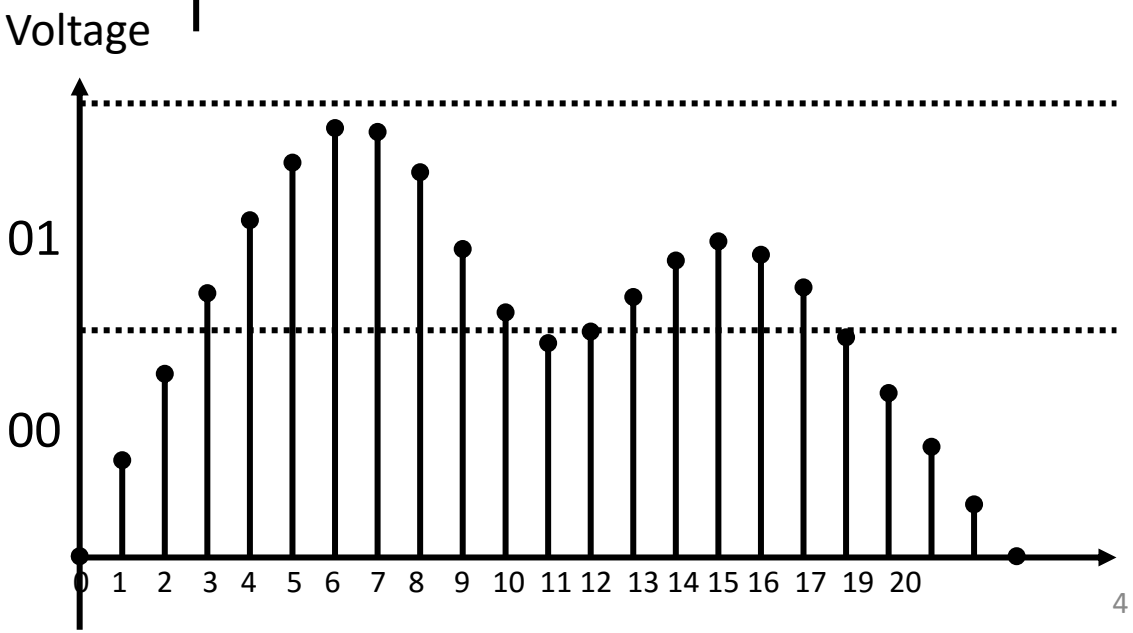
Binary Encoding

Digital Instrumentation (ADC Vres)

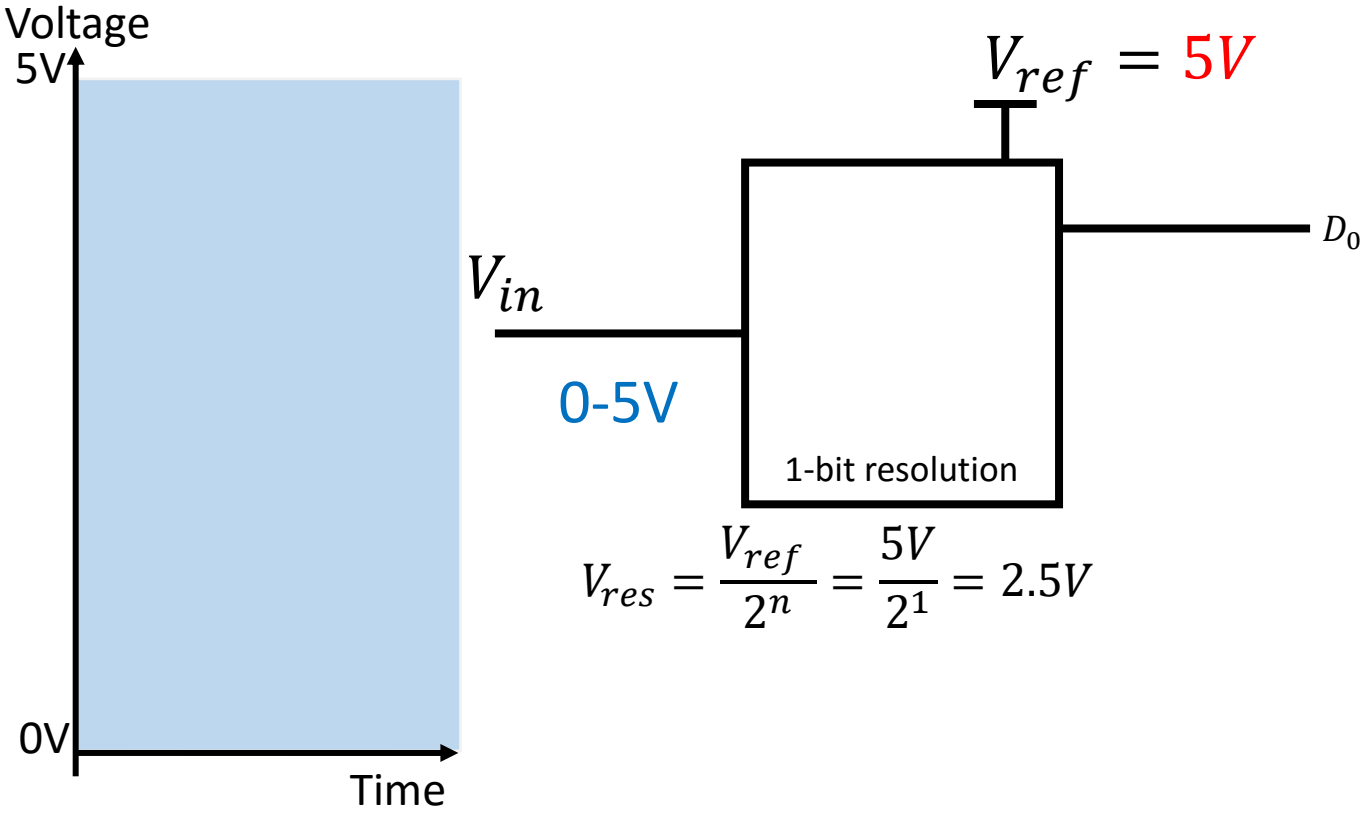


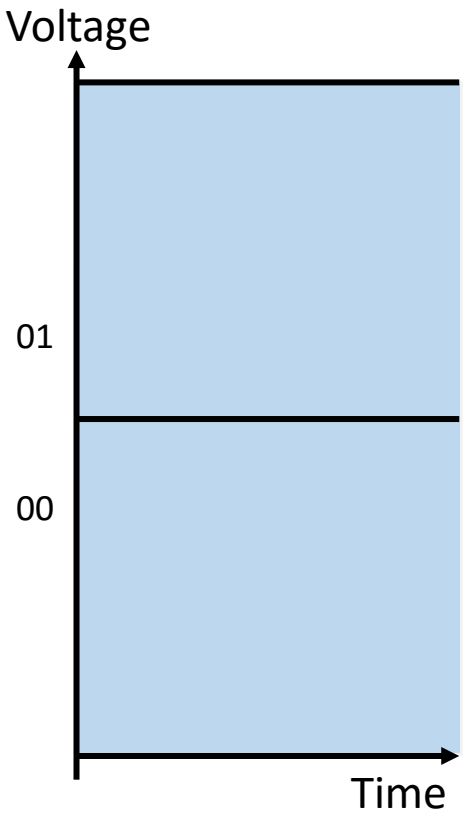






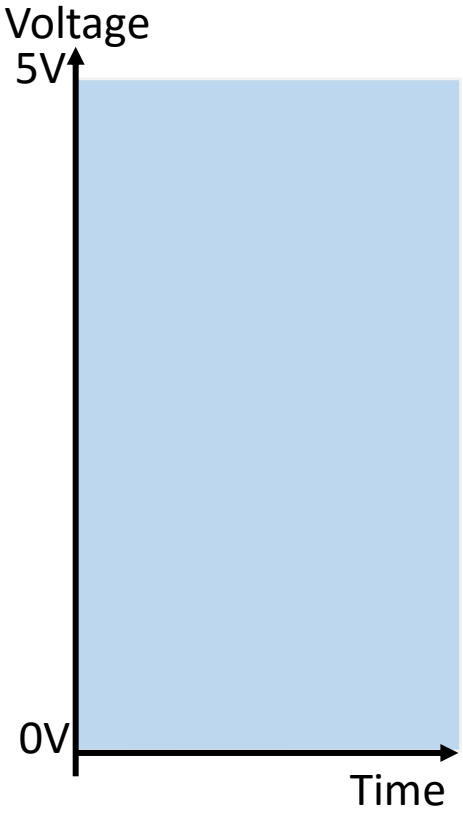
ADC Basics (n-bit resolution effect on Vres)

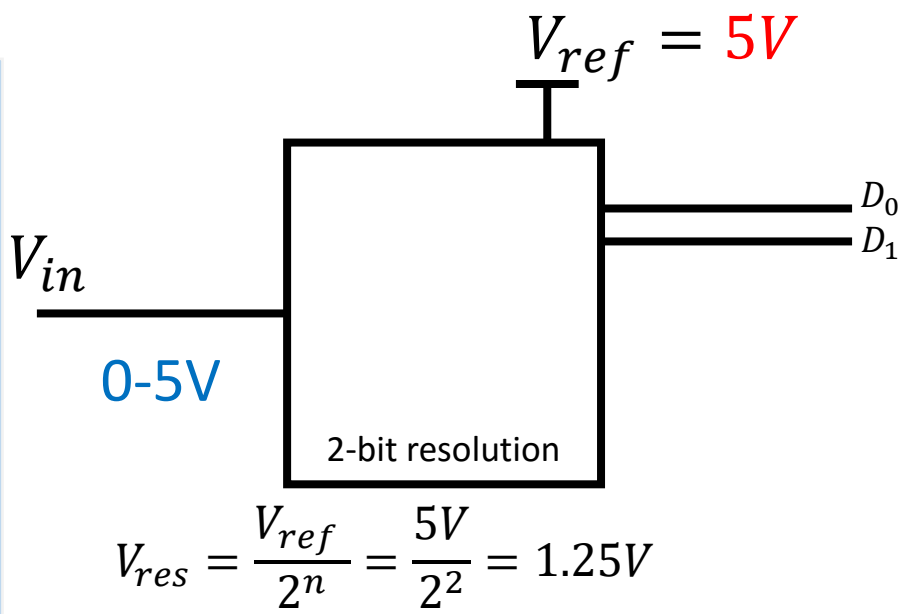




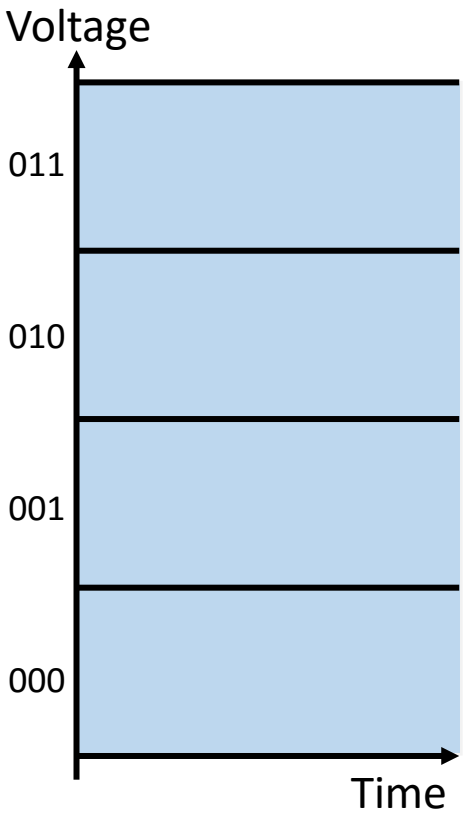
Vin	ADCout (B)	ADCout (D)
0 ≤ Vin < 2.5V	0	0
2.5V ≤ Vin < 5V	1	1

ADC Basics (n-bit resolution effect on Vres)



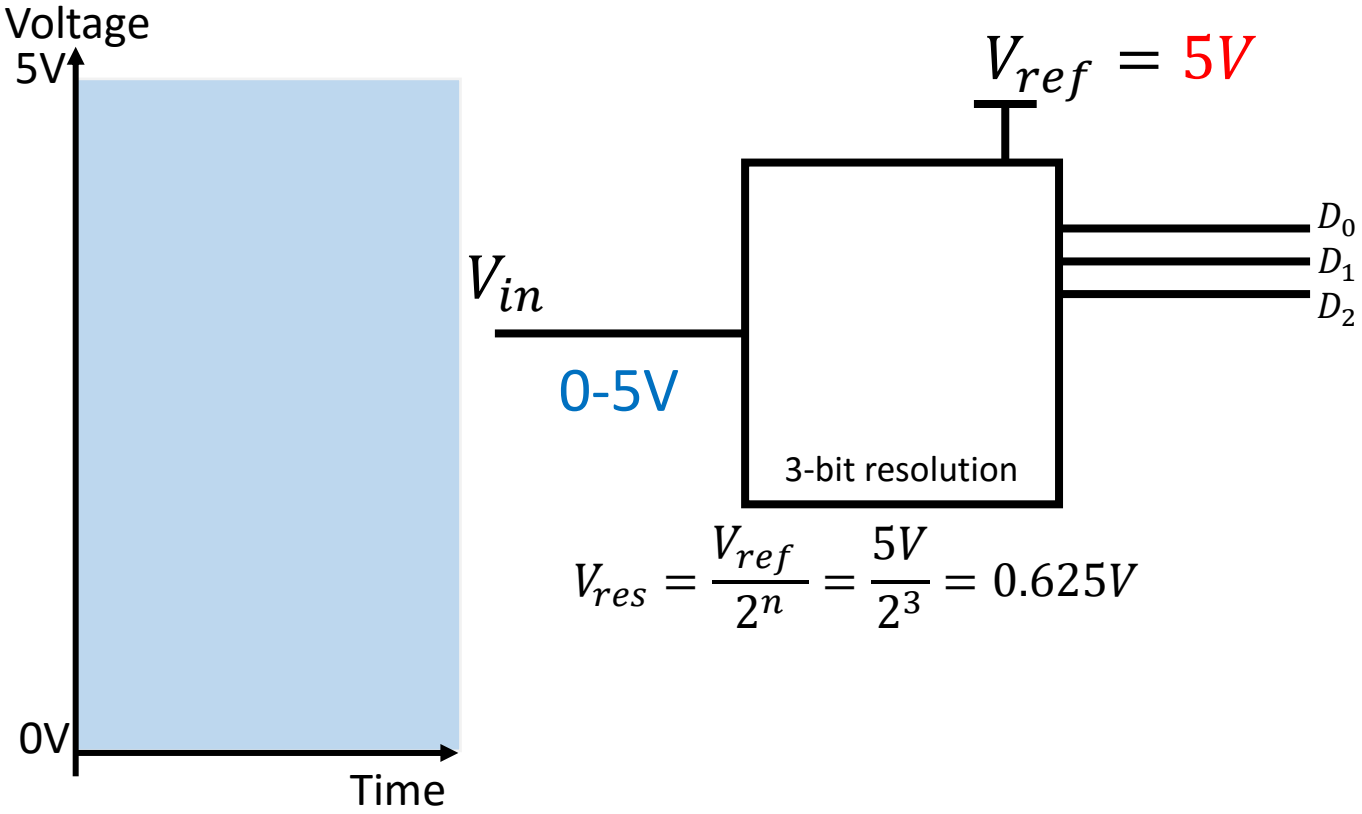


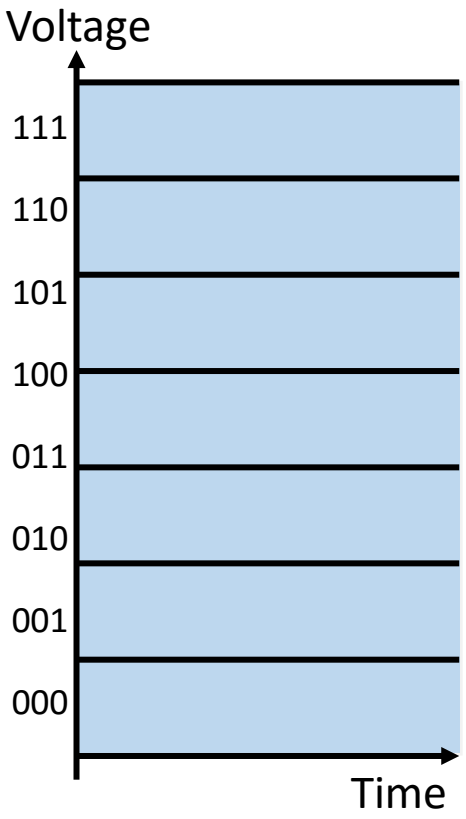
$$V_{res} = \frac{V_{ref}}{2^n} = \frac{5V}{2^2} = 1.25V$$



Vin	ADCout (B)	ADCout (D)
0 ≤ Vin < 1.25V	00	0
1.25V ≤ Vin < 2.5V	01	1
2.5V ≤ Vin < 3.75V	10	2
3.75V ≤ Vin < 5V	11	3

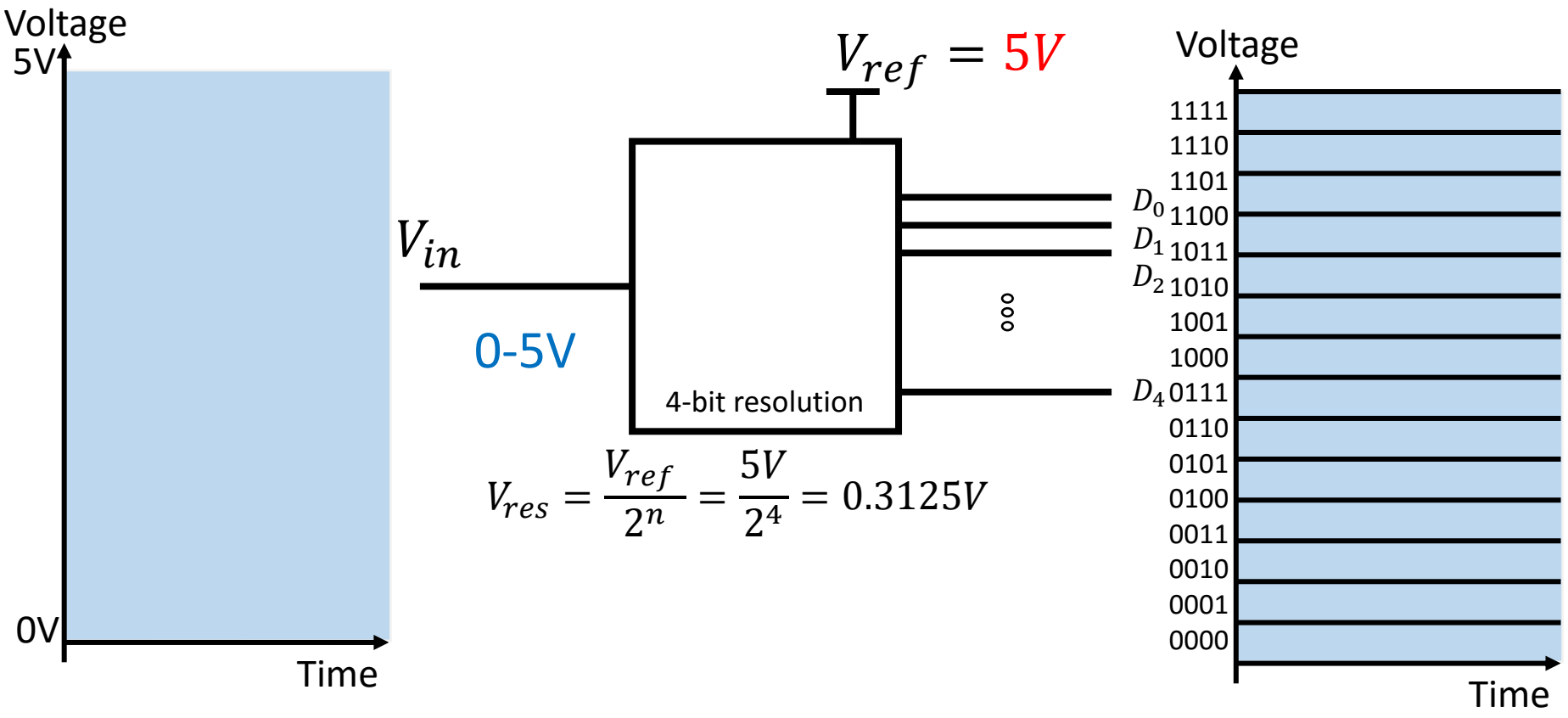
ADC Basics (n-bit resolution effect on Vres)





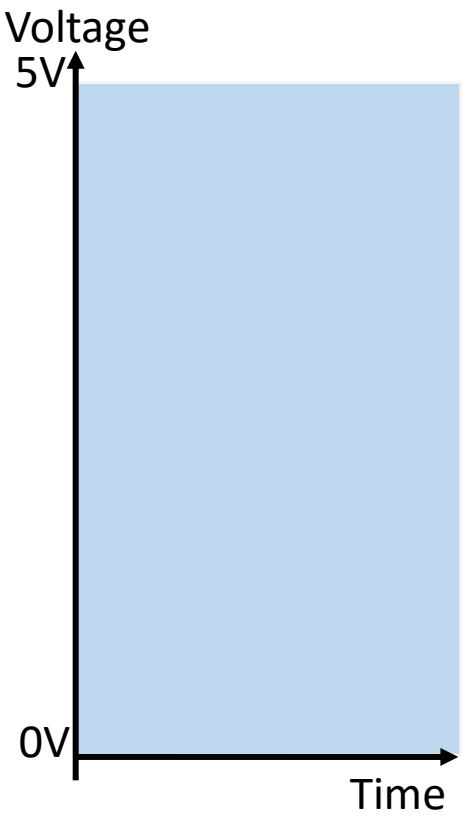
Vin	ADCout (B)	ADCout (D)
0 ≤ Vin < 0.625V	000	0
0.625V ≤ Vin < 1.25V	001	1
1.25V ≤ Vin < 1.875V	010	2
1.875V ≤ Vin < 2.5V	011	3
2.5V ≤ Vin < 3.125V	100	4
3.125V ≤ Vin < 3.75V	101	5
3.75V ≤ Vin < 4.375V	110	6
4.375 ≤ Vin < 5V	111	7

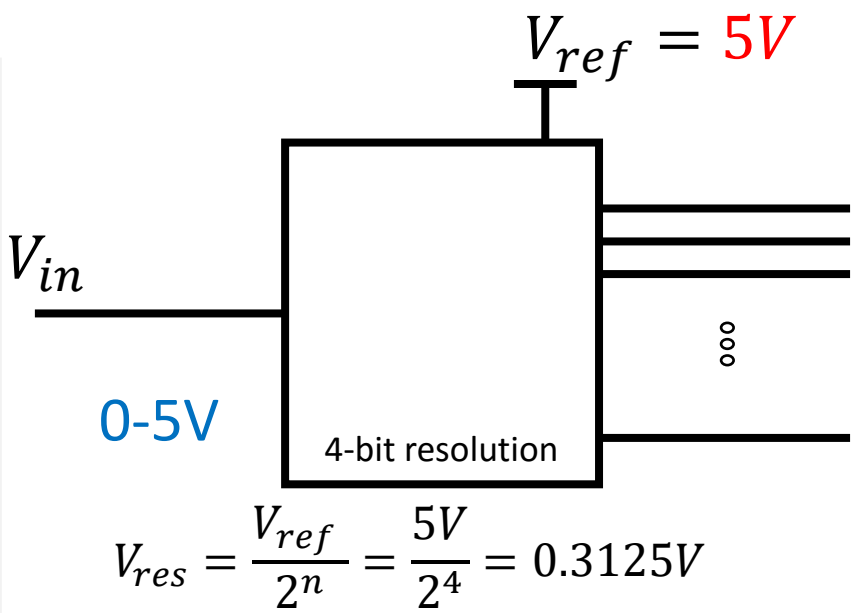
ADC Basics (n-bit resolution effect on Vres)



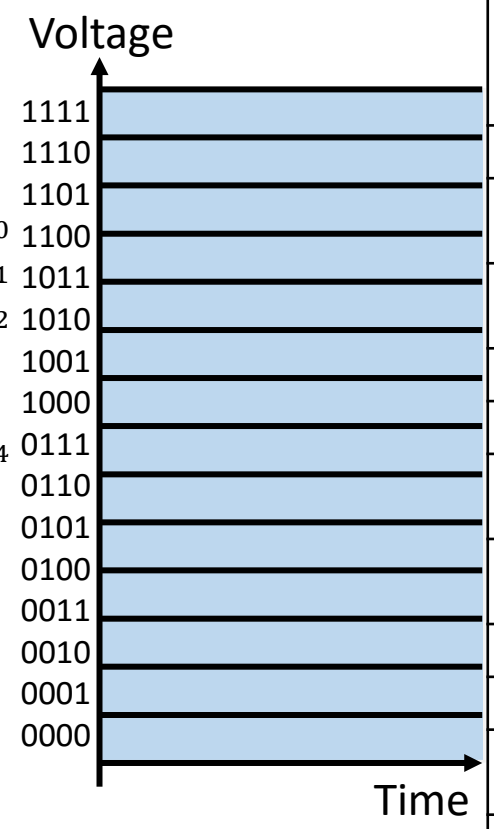
Vin	ADCout (B)	ADCout (D)
0 ≤ Vin < 0.3125V	0000	0
0.3125V ≤ Vin < 0.625V	0001	1
0.625V ≤ Vin < 0.9375V	0010	2
0.9375V ≤ Vin < 1.25V	0011	3
1.25V ≤ Vin < 1.5625V	0100	4
1.5625V ≤ Vin < 1.875V	0101	5
1.875V ≤ Vin < 2.1875V	0110	6
2.1875 ≤ Vin < 2.5V	0111	7
2.5V ≤ Vin < 2.8125V	1000	8
2.8125V ≤ Vin < 3.125V	1001	9
3.125V ≤ Vin < 3.4375V	1010	10
3.4375 ≤ Vin < 3.75V	1011	11
3.75V ≤ Vin < 4.0625V	1100	12
4.0625V ≤ Vin < 4.375V	1101	13
4.375V ≤ Vin < 4.6875V	1110	14
4.6875V ≤ Vin < 5V	1111	15

ADC Basics (Vref effect on Vres)



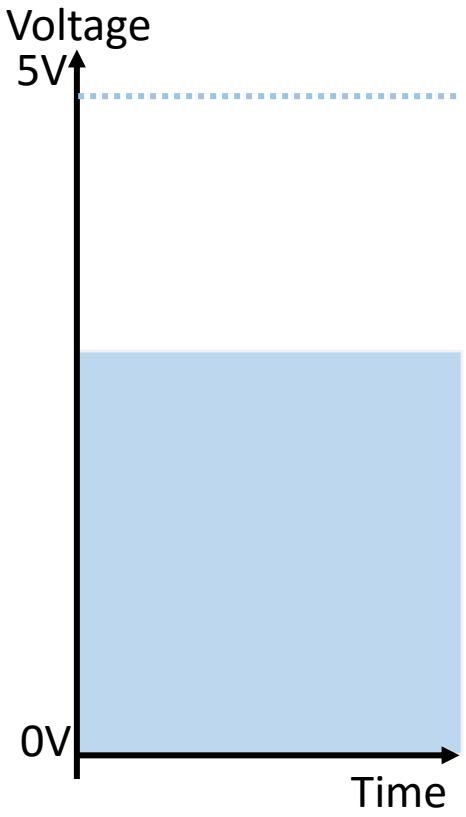


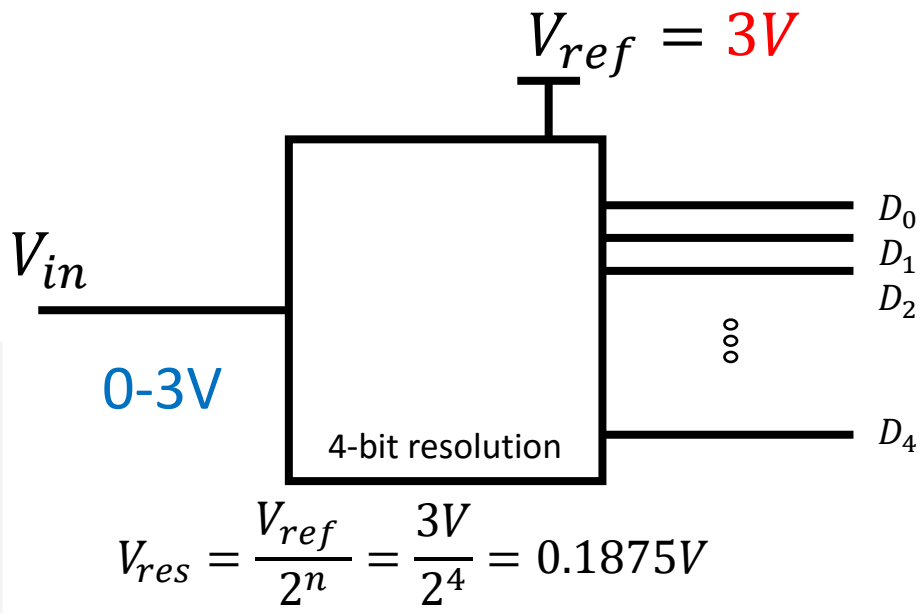
$$V_{res} = \frac{V_{ref}}{2^n} = \frac{5V}{2^4} = 0.3125V$$



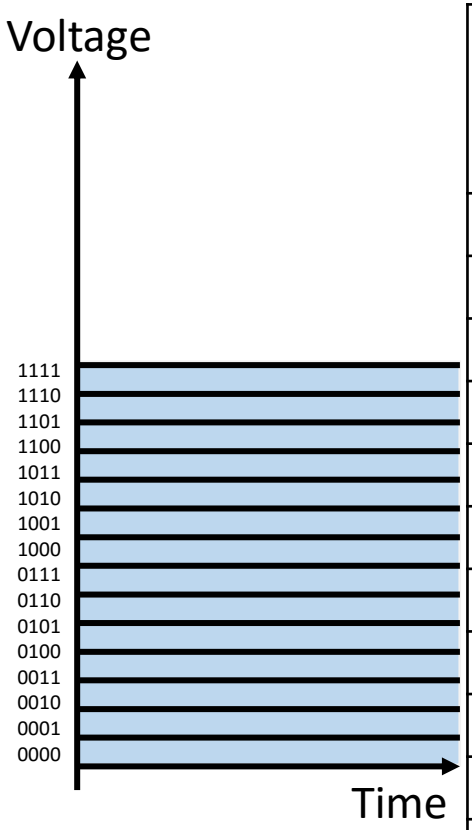
Vin	ADCout (B)	ADCout (D)
0 ≤ Vin < 0.3125V	0000	0
0.3125V ≤ Vin < 0.625V	0001	1
0.625V ≤ Vin < 0.9375V	0010	2
0.9375V ≤ Vin < 1.25V	0011	3
1.25V ≤ Vin < 1.5625V	0100	4
1.5625V ≤ Vin < 1.875V	0101	5
1.875V ≤ Vin < 2.1875V	0110	6
2.1875 ≤ Vin < 2.5V	0111	7
2.5V ≤ Vin < 2.8125V	1000	8
2.8125V ≤ Vin < 3.125V	1001	9
3.125V ≤ Vin < 3.4375V	1010	10
3.4375 ≤ Vin < 3.75V	1011	11
3.75V ≤ Vin < 4.0625V	1100	12
4.0625V ≤ Vin < 4.375V	1101	13
4.375V ≤ Vin < 4.6875V	1110	14
4.6875V ≤ Vin < 5V	1111	15

ADC Basics (Vref effect on Vres)



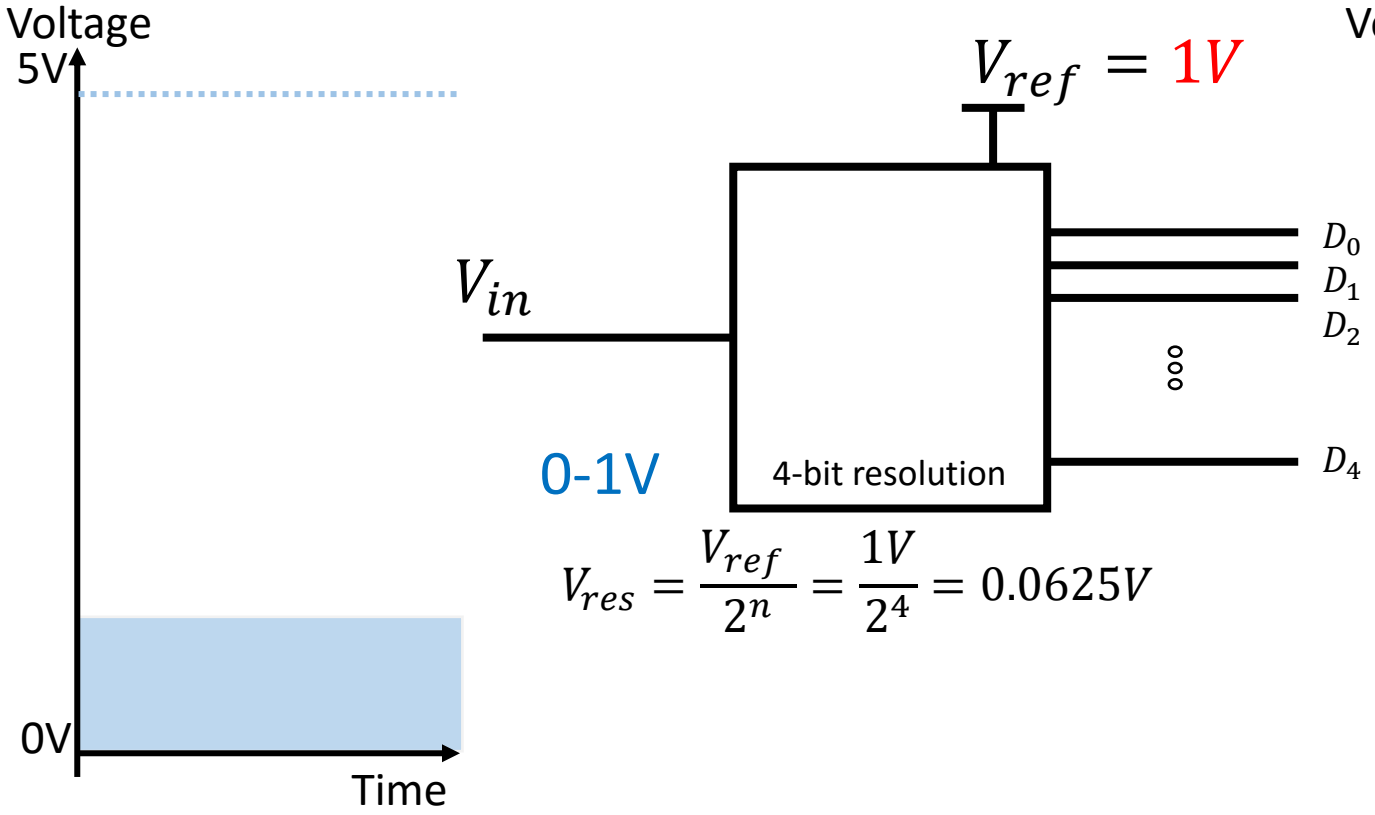


$$V_{res} = \frac{V_{ref}}{2^n} = \frac{3V}{2^4} = 0.1875V$$

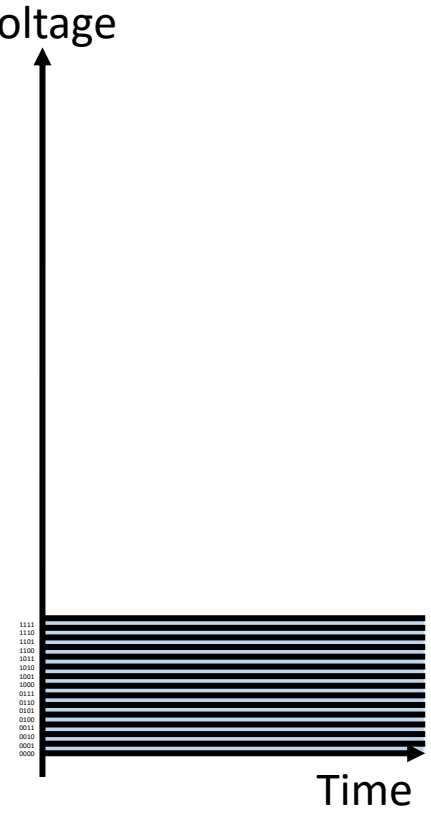


Vin	ADCout (B)	ADCout (D)
0V ≤ Vin < 0.1875V	0000	0
0.1875V ≤ Vin < 0.375V	0001	1
0.375V ≤ Vin < 0.5625V	0010	2
0.5625V ≤ Vin < 0.75V	0011	3
0.75V ≤ Vin < 1.9375V	0100	4
1.9375V ≤ Vin < 1.125V	0101	5
1.125V ≤ Vin < 1.3125V	0110	6
1.3125V ≤ Vin < 1.5V	0111	7
1.5V ≤ Vin < 1.6875V	1000	8
1.6875V ≤ Vin < 1.875V	1001	9
1.875V ≤ Vin < 2.0625V	1010	10
2.0625V ≤ Vin < 2.25V	1011	11
2.25V ≤ Vin < 2.4375V	1100	12
2.4375V ≤ Vin < 2.6250V	1101	13
2.6250V ≤ Vin < 2.8125	1110	14
2.8125 ≤ Vin < 3.0	1111	15

ADC Basics (Vref effect on Vres)



$$V_{res} = \frac{V_{ref}}{2^n} = \frac{1V}{2^4} = 0.0625V$$



Vin	ADCout (B)	ADCout (D)
0V ≤ Vin < 0.0625V	0000	0
0.0625V ≤ Vin < 0.125V	0001	1
0.125V ≤ Vin < 0.1875V	0010	2
0.1875V ≤ Vin < 0.25V	0011	3
0.25V ≤ Vin < 0.3125V	0100	4
0.3125V ≤ Vin < 0.375V	0101	5
0.375V ≤ Vin < 0.4375V	0110	6
0.4375V ≤ Vin < 0.5V	0111	7
0.5V ≤ Vin < 0.5625V	1000	8
0.5625V ≤ Vin < 0.625V	1001	9
0.625V ≤ Vin < 0.6875V	1010	10
0.6875V ≤ Vin < 0.75V	1011	11
0.75V ≤ Vin < 0.8125V	1100	12
0.8125V ≤ Vin < 0.875V	1101	13
0.875V ≤ Vin < 0.9375V	1110	14
0.9375V ≤ Vin < 1.0	1111	15

Resolution:

ADC has n-bit resolution, where n can be 8,10, 12, 16 or 24 bits.

- Higher resolution provides a smaller step size
- STEP SIZE is the smallest change that can be detected by an ADC.
- ADC chip resolution is decided at the time of design (fabrication) can cannot be changed.
- Step Size can be controlled with the Vref.

n-bit	Number of steps	Step size (mV)
8	256	5/256 = 19.53
10	1024	5/1024 = 4.88
12	4096	5/4096 = 1.2
16	65,536	5/65,536 = 0.076

Conversion Time:

The time ADC takes to convert the analog input to a digital (binary) number. Conversion time depends upon:

- Clock source connected to ADC
- Method use for data conversion (Flash, Sigma-Delta, Successive approximation converter)
- Technology used in the fabrication (MOS or TTL)

Vref:

- An input voltage used for the reference voltage, it selects input voltage range
- Voltage connected to this pin dictates the step size
- n-bit ADC, step size = Vref / n-bit resolution
- For 10-bit ADC, step size = Vref / 1024

Example: IF analog input range needs to be 0V to 4V, Vref is connected to 4V.

- Step Size = 4V/1024 = 3.91mV.
- If we need a step size of 2.5mV, then Vref = 2.56V because 2.56/1024 = 2.5mV.
- In some application we need differential voltage where Vref = Vref (+) – Vref (–).
- Often the Vref (–) pin is connected to ground and Vref (+) pin is used as the Vref.

V_{ref} (V)	V_{in} (V)	Step Size (mV)
5.00	0 to 5	5/1024 = 4.88
4.096	0 to 4.096	4.096/1024 = 4
3.0	0 to 3	3/1024 = 2.93
2.56	0 to 2.56	2.56/1024 = 2.5
2.048	0 to 2.048	2.048/1024 = 2
1.28	0 to 1.28	1/1024 = 1.25
1.024	0 to 1.024	1.024/1024 = 1

Digital Data Output:

In 10-bit ADC, the data output is D9 – D0. To calculate the output voltage:

$$Dout = Vin / step\ size$$

Example: For 10-bit ADC and Vref = 2.56V. Calculate the D9 – D0 output if the analog input is:
(a) 1.7V (b) 2.1V and (c) 0.5V

Step size = 2.56/1024 = 2.5mV

So,

(a) Dout = 1.7V / 2.5mV = 680
(b) Dout = 2.1V / 2.5mV = 840
(c) Dout = 0.5V / 2.5mV = 200

Analog Input Channels:

Many data acquisition applications need more than one ADC. For this reason ADC chips has 2, 4, 8, or even 16 channel on a single chip.

Start-Conversion and End-of –Conversion Signals:

- When SC is activated, ADC starts converting the analog input of Vin to n-bit digital number.
- EoC signals the CPU that data conversion is complete and converted data is available to pick-up.

The **ADC peripherals of Mega2560** has the following **characteristics**:

1. It is a **10-bit ADC**
2. It has **16 analog channels**, 14 differential input channels and 4 differential input channels with gain of 10x and 200x
3. The converted data is held by two registers called **ADCL and ADCH (16bit register ADC)**
4. ADCH:ADCL registers are 16-bit and ADC data out is only 10-bit wide. 6-bits are unused (selection option making upper 6-bits or the lower 6-bits unused)
5. **Four options for Vref**.
   A. Vref can be connected to AVCC (Analog Vcc)
   B. Internal 1.1V reference
   C. Internal 2.56V reference
   D. External AREF pin
6. Conversion time is dictated by the crystal frequency connected to XTAL pins (FOSC) and ADPS2:0 bits
7. **Input clock frequency between 50kHz and 200kHz**
8. 13µs - 260µs Conversion Time

ADC Registers and Programming

Five major **registers** are used with ADC. They are:

- ADCH
- ADCL

} **ADC**

- ADMUX
- ADCSRA
- ADCSRB

ADMUX – ADC Multiplexer Selection Register:

Bit	7	6	5	4	3	2	1	0	
(0x7C)	REFS1	REFS0	ADLAR	MUX4	MUX3	MUX2	MUX1	MUX0	ADMUX
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7:6 – REFS1:0: Reference Selection Bits**

These bits select the voltage reference for the ADC. If these bits are changed during a conversion, the change will not go in effect until this conversion is complete (ADIF in ADCSRA is set). The internal voltage reference options may not be used if an external reference voltage is being applied to the AREF pin.

REFS1	REFS0	Voltage Reference Selection[1]
0	0	AREF, Internal V_{REF} turned off
0	1	AVCC with external capacitor at AREF pin
1	0	Internal 1.1V Voltage Reference with external capacitor at AREF pin
1	1	Internal 2.56V Voltage Reference with external capacitor at AREF pin

- **Bit 5 – ADLAR: ADC Left Adjust Result**

The ADLAR bit affects the presentation of the ADC conversion result in the ADC Data Register. Write one to ADLAR to left adjust the result. Otherwise, the result is right adjusted. Changing the ADLAR bit will affect the ADC Data Register immediately, regardless of any ongoing conversions.

- **Bits 4:0 – MUX4:0: Analog Channel and Gain Selection Bits**

The value of these bits selects which combination of analog inputs are connected to the ADC. If these bits are changed during a conversion, the change will not go in effect until this conversion is complete (ADIF in ADCSRA is set).

MUX5:0	Single Ended Input	Positive Differential Input	Negative Differential Input	Gain
000000	ADC0	N/A		
000001	ADC1			
000010	ADC2			
000011	ADC3			
000100	ADC4			
000101	ADC5			
000110	ADC6			
000111	ADC7			
100000	ADC8	N/A		
100001	ADC9			
100010	ADC10			
100011	ADC11			
100100	ADC12			
100101	ADC13			
100110	ADC14			
100111	ADC15			

MUX5:0	Single Ended Input	Positive Differential Input	Negative Differential Input	Gain
001000[1]	N/A	ADC0	ADC0	10×
001001[1]		ADC1	ADC0	10×
001010[1]		ADC0	ADC0	200×
001011[1]		ADC1	ADC0	200×
001100[1]		ADC2	ADC2	10×
001101[1]		ADC3	ADC2	10×
001110[1]		ADC2	ADC2	200×
001111[1]		ADC3	ADC2	200×
010000		ADC0	ADC1	1×
010001		ADC1	ADC1	1×
010010		ADC2	ADC1	1×
010011		ADC3	ADC1	1×
010100		ADC4	ADC1	1×
010101		ADC5	ADC1	1×
010110		ADC6	ADC1	1×
010111		ADC7	ADC1	1×
011000		ADC0	ADC2	1×
011001		ADC1	ADC2	1×
011010	N/A	ADC2	ADC2	1×
011011		ADC3	ADC2	1×
011100		ADC4	ADC2	1×
011101		ADC5	ADC2	1×
011110	1.1V (V_{BG})	N/A		
011111	0V (GND)			

ADCSRA – ADC Control and Status Register A:

Bit	7	6	5	4	3	2	1	0	
(0x7A)	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0	ADCSRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – ADEN: ADC Enable**

Writing this bit to one enables the ADC. By writing it to zero, the ADC is turned off. Turning the ADC off while a conversion is in progress, will terminate this conversion.

- **Bit 6 – ADSC: ADC Start Conversion**

In Single Conversion mode, write this bit to one to start each conversion. In Free Running mode, write this bit to one to start the first conversion. The first conversion after ADSC has been written after the ADC has been enabled, or if ADSC is written at the same time as the ADC is enabled, will take 25 ADC clock cycles instead of the normal 13. This first conversion performs initialization of the ADC. ADSC will read as one as long as a conversion is in progress. When the conversion is complete, it returns to zero. Writing zero to this bit has no effect.

- **Bit 5 – ADATE: ADC Auto Trigger Enable**

When this bit is written to one, Auto Triggering of the ADC is enabled. The ADC will start a conversion on a positive edge of the selected trigger signal. The trigger source is selected by setting the ADC Trigger Select bits, ADTS in ADCSRB.

- **Bit 4 – ADIF: ADC Interrupt Flag**

This bit is set when an ADC conversion completes and the Data Registers are updated. The ADC Conversion Complete Interrupt is executed if the ADIE bit and the I-bit in SREG are set. ADIF is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, ADIF is cleared by writing a logical one to the flag. Beware that if doing a Read-Modify-Write on ADCSRA, a pending interrupt can be disabled. This also applies if the SBI and CBI instructions are used.

- **Bit 3 – ADIE: ADC Interrupt Enable**

When this bit is written to one and the I-bit in SREG is set, the ADC Conversion Complete Interrupt is activated.

- **Bits 2:0 – ADPS2:0: ADC Prescaler Select Bits**

These bits determine the division factor between the XTAL frequency and the input clock to the ADC.

ADC circuitry requires an input clock frequency between 50kHz and 200kHz. ADC_Freq = $^{Fc}/_{Division\ Factor}$

ADPS2	ADPS1	ADPS0	Division Factor
0	0	0	2
0	0	1	2
0	1	0	4
0	1	1	8
1	0	0	16
1	0	1	32
1	1	0	64
1	1	1	128

ADCL and ADCH – The ADC Data Register:

ADLAR = 0

Bit	15	14	13	12	11	10	9	8	
(0x79)	–	–	–	–	–	–	ADC9	ADC8	ADCH
(0x78)	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADC1	ADC0	ADCL
	7	6	5	4	3	2	1	0	

ADLAR = 1

Bit	15	14	13	12	11	10	9	8	
(0x79)	ADC9	ADC8	ADC7	ADC6	ADC5	ADC4	ADC3	ADC2	ADCH
(0x78)	ADC1	ADC0	–	–	–	–	–	–	ADCL
	7	6	5	4	3	2	1	0	

ADCSRB – ADC Control and Status Register B:

Bit	7	6	5	4	3	2	1	0	
(0x7B)	–	ACME	–	–	MUX5	ADTS2	ADTS1	ADTS0	ADCSRB
Read/Write	R	R/W	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – Res: Reserved Bit**

This bit is reserved for future use. To ensure compatibility with future devices, this bit must be written to zero when ADCSRB is written.

- **Bit 3 – MUX5: Analog Channel and Gain Selection Bit**

This bit is used together with MUX4:0 in ADMUX to select which combination in of analog inputs are connected to the ADC. If this bit is changed during a conversion, the change will not go in effect until this conversion is complete.

- **Bit 2:0 – ADTS2:0: ADC Auto Trigger Source**

If ADATE in ADCSRA is written to one, the value of these bits selects which source will trigger an ADC conversion. If ADATE is cleared, the ADTS2:0 settings will have no effect. A conversion will be triggered by the rising edge of the selected Interrupt Flag. Note that switching from a trigger source that is cleared to a trigger source that is set, will generate a positive edge on the trigger signal. If ADEN in ADCSRA is set, this will start a conversion. Switching to Free Running mode (ADTS[2:0]=0) will not cause a trigger event, even if the ADC Interrupt Flag is set.

ADTS2	ADTS1	ADTS0	Trigger Source
0	0	0	Free Running mode
0	0	1	Analog Comparator
0	1	0	External Interrupt Request 0
0	1	1	Timer/Counter0 Compare Match A
1	0	0	Timer/Counter0 Overflow
1	0	1	Timer/Counter1 Compare Match B
1	1	0	Timer/Counter1 Overflow
1	1	1	Timer/Counter1 Capture Event

Steps in programming the ADC:

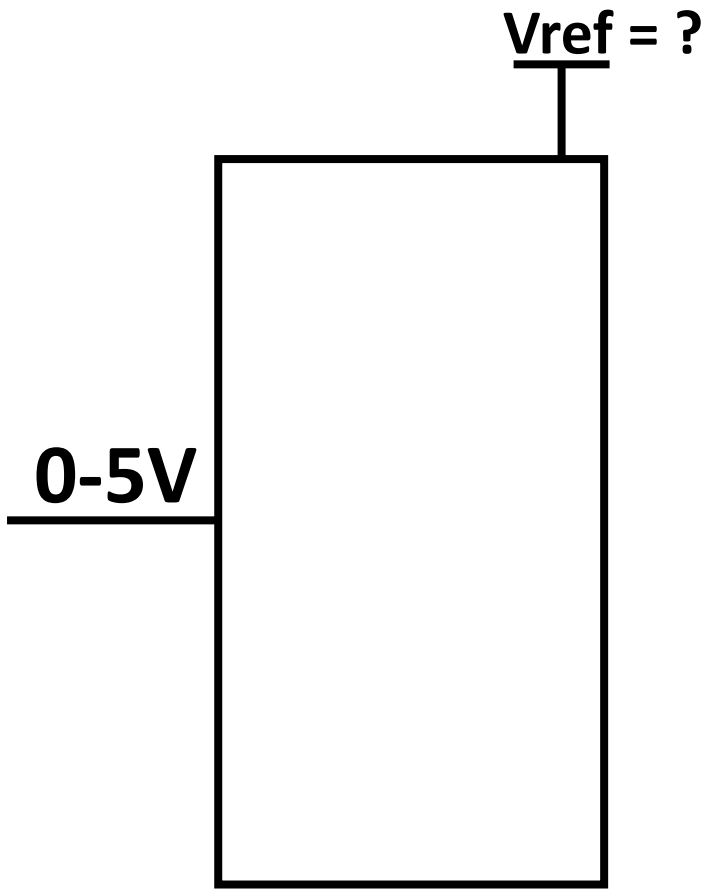
1. Make the pin for the selected ADC channel an input pin.
2. Turn ON the ADC module.
3. Select the conversion speed by using register ADCSRA bits ADPS2:0
4. Select voltage reference and ADC input channel. Use ADMUX bits, REFS1:0 for voltage reference and MUX5:0 to select the ADC input channel.
5. Activate the start conversion bit by writing a one to the ADSC bit of ADSCRA register.
6. Wait for the conversion to be completed by polling ADIF bit in the ADCSRA register.
7. After ADIF bit has gone HIGH, read the ADCL and ADCH register to get the digital data output. Read ADCL before ADCH.

Write a program to get data from channel 0 (ADC0) of ADC and display the results on PORTK and PORTE , continuously.

```
#include<avr/io.h>
int main(void)
{
	DDRF = 0; // make PORTF an input for ADC0 input
	DDRK = 0xFF;
	DDRE = 0xFF;
	ADCSRA = 0x87;
	ADCSRB = 0x00;
	ADMUX = 0xC0;
	while(1)
	{
		ADCSRA |= (1<<ADSC);
		while(ADCSRA & (1<< ADIF) ==0);
		PORTK = ADCL;
		PORTE = ADCH;
	}
	return 0;
}
```

Design a digital voltmeter range 0-5V.

```
#include<avr/io.h>
int main(void)
{
	DDRF = 0; // make PORTF an input for ADC0 input
	ADCSRA = 0x87;
	ADCSRB = 0x00;
	ADMUX = 0xC0;
	while(1)
	{
		ADCSRA |= (1<<ADSC);
		while(ADCSRA & (1<< ADIF) ==0);
		PORTK = ADCL/?;
	}
	return 0;
}
```

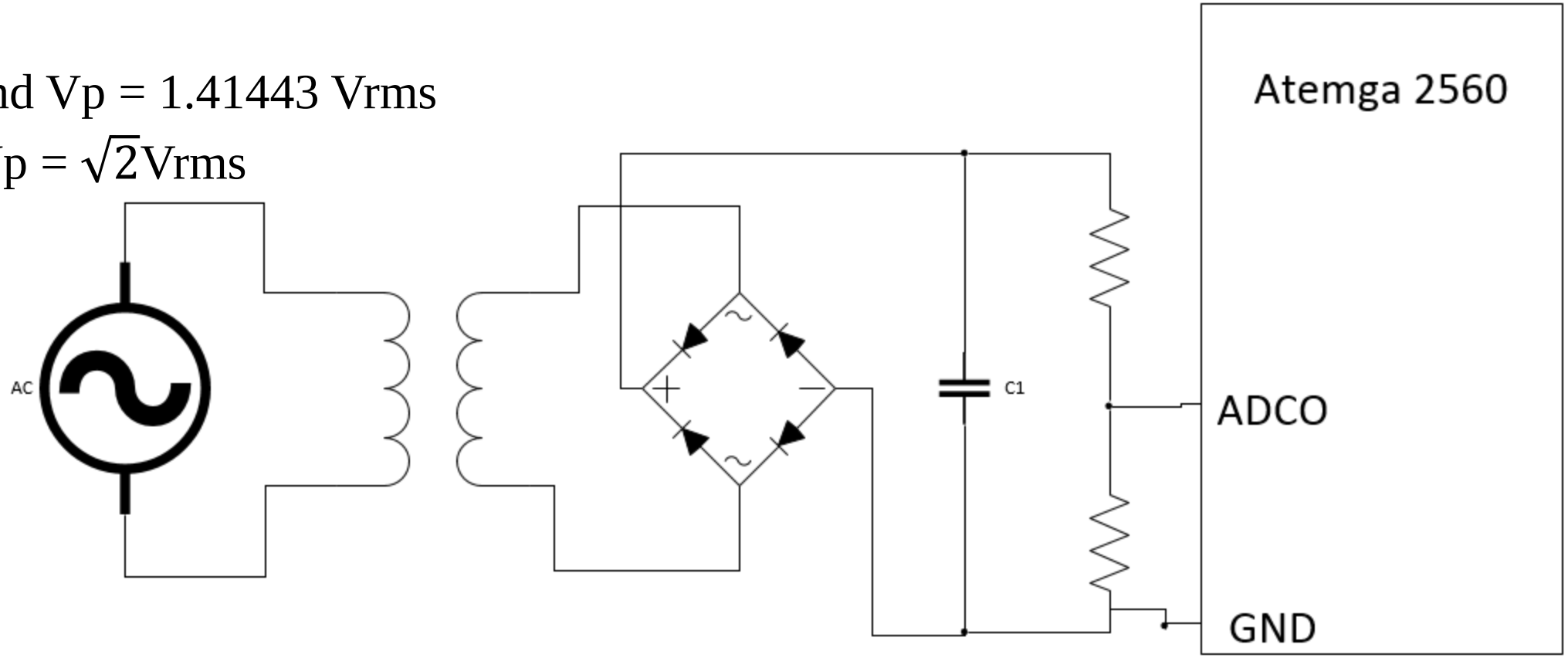


```
#include<avr/io.h>
int main(void)
{
 DDRF |= ~(1<<6) ; // make PORTF an input for ADC0 input
 DDRK = 0xFF;
 DDRB = 0xFF;
 ADMUX = 0x46;
 ADCSRA = 0x87;
 ADCSRB = 0x00;
 Serial.begin(9600);
 float analog_value;
 while(1)
 {
  ADCSRA |= (1<<ADSC);
  while((ADCSRA & (1<< ADIF)) ==0);
  analog_value = (ADC * 5.0) /1023.0;
  PORTK = analog_value;
  Serial.println(analog_value);
 }
 return 0;
}
```

AC Voltmeter:

Vrms = 0.707 Vp and Vp = 1.41443 Vrms

Vrms = $\frac{1}{\sqrt{2}}$Vp and Vp = $\sqrt{2}$Vrms

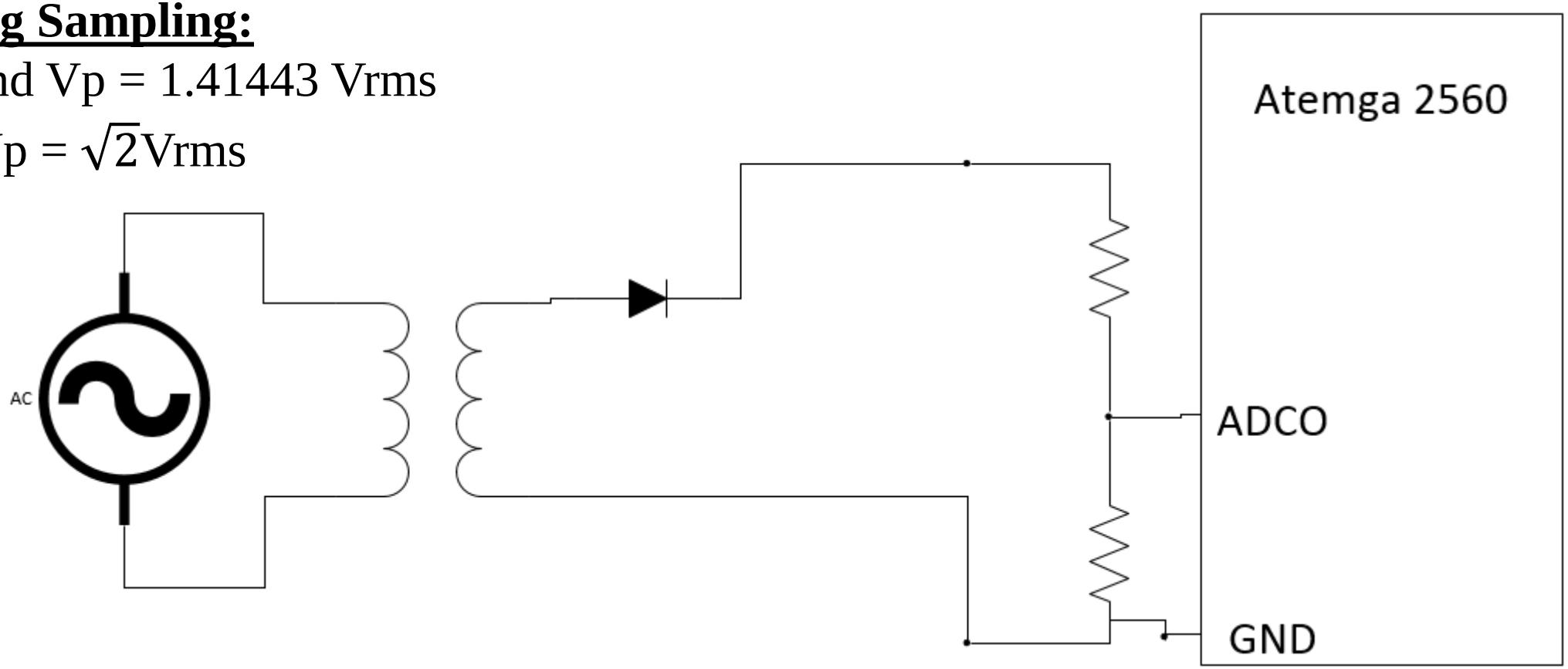


$$AC\ Voltmeter\ Read = \left[\left[\left[\left[ADCin * \frac{5}{1023}\right] * \frac{21.745}{5}\right] + 1.4\right] * 18.33\right] * 0.707$$

AC Voltmeter using Sampling:

Vrms = 0.707 Vp and Vp = 1.41443 Vrms

Vrms = $\frac{1}{\sqrt{2}}$Vp and Vp = $\sqrt{2}$Vrms



F = 50Hz

T = 20msec

FADC = 125KHz

TADC = 8µsec

Total samples in one clock = $\frac{20ms}{8\mu sec} = 2500$

- ✓ Store 2500 samples, i.e, x[2500]
- ✓ Find zero crossing, i.e, if(ADC==0)
- ✓ For (i=0,i<2500;i++)

```
{
x[i] = ADC;
}
```

- ✓ maxvalue = max(x)

$$AC\ Voltmeter\ Read = \left[\left[\left[\left[maxvalue * \frac{5}{1023}\right] * \frac{22.445}{5}\right] + 0.7\right] * 18.33\right] * 0.707$$

Examples ADC

- Write a Code for designing a DC Voltmeter of range 0-5v.
- Write a Code to measure DC voltages from 0-30v.
- Design a temperature display unit, which displays temperature by using LM 35 Sensor between 0°C-75°C.
- Design a temperature monitoring system for chicken hatchery. You are required to buzzer the alarm when temperature is raised from the set point, and also, Open and Close the door using DC motor when specified temperature criteria is meet.

IF Temperature is between 37°C-43°C	Open the Door
IF Temperature is above 45°C	Buzzer the alarm with 1000Hz signal.

Sample-and-Hold time in ADC:

After an ADC channel is selected, the ADC allows some time for the sample-and-hold capacitor to charge fully to the input voltage level present at the channel.

Condition	Sample & Hold (Cycles from Start of Conversion)	Conversion Time (Cycles)
First conversion	13.5	25
Normal conversions, single ended	1.5	13
Auto Triggered conversions	2	13.5
Normal conversions, differential	1.5/2.5	13/14

Digital Input Disable Registers

DIDR0 – Digital Input Disable Register 0:

Bit	7	6	5	4	3	2	1	0	
(0x7E)	ADC7D	ADC6D	ADC5D	ADC4D	ADC3D	ADC2D	ADC1D	ADC0D	DIDR0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7:0 – ADC7D:ADC0D: ADC7:0 Digital Input Disable**

When this bit is written logic one, the digital input buffer on the corresponding ADC pin is disabled. The corresponding PIN Register bit will always read as zero when this bit is set. When an analog signal is applied to the ADC7:0 pin and the digital input from this pin is not needed, this bit should be written logic one to reduce power consumption in the digital input buffer.

DIDR2 – Digital Input Disable Register 2:

Bit	7	6	5	4	3	2	1	0	
(0x7D)	ADC15D	ADC14D	ADC13D	ADC12D	ADC11D	ADC10D	ADC9D	ADC8D	DIDR2
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7:0 – ADC15D:ADC8D: ADC15:8 Digital Input Disable**

When this bit is written logic one, the digital input buffer on the corresponding ADC pin is disabled. The corresponding PIN Register bit will always read as zero when this bit is set. When an analog signal is applied to the ADC15:8 pin and the digital input from this pin is not needed, this bit should be written logic one to reduce power consumption in the digital input buffer.

```
#include<avr/io.h>
int main(void)
{
  DDRF |= ~(1<<6) ; // make PORTF an input for ADC0 input
  DDRK = 0xFF;
  ADMUX = 0xC6;
  ADCSRA = 0x87;
  ADCSRB = 0x00;
  Serial.begin(9600);
  unsigned char analog_value;
  while(1)
  {
    ADCSRA |= (1<<ADSC);
    while((ADCSRA & (1<< ADIF)) ==0);
    analog_value = (ADC) /4;
    Serial.println(analog_value);
  }
  return 0;
}
```

```
#include<avr/io.h>
int main(void)
{
  DDRF |= ~(1<<6) ; // make PORTF an input for ADC0 input
  DDRK |= (1<<7);
  ADMUX = 0xC6;
  ADCSRA = 0x87;
  ADCSRB = 0x00;
  Serial.begin(9600);
  unsigned char B;
  while(1)
  {
    ADCSRA |= (1<<ADSC);
    while((ADCSRA & (1<< ADIF)) ==0);
    B = (ADC) /4;
    PORTK = B;
    Serial.println(B);
    if (B == 30)
      PORTK |= (1<<7);
    else
      PORTK &= ~(1<<7);
  }
  return 0;
}
```